The Vapnik Chervonenkis Bound

Slide 2 The famous Hoeffding Equation

In this module we will explore the theory of learning and an important corner stone of this theory is the Hoeffding Bound. This Hoeffding bound is a general result from probability theory that provides us with an upper bound on the likely sample error.

What do we mean by this? Suppose we have a jar, or bin with coloured marbles. We cannot see the marbles in this jar and we would like to find the fraction of blue marbles, or in other words the probability of picking a blue marble from the jar. Let's call this probability mu.

You will agree with me that a sensible way of finding mu experimentally is by picking a number of marbles and noting their colour before replacing them. We will call the found fraction of blue marbles in our sample, nu. Clearly, picking one marble will not be enough, 2 will also give a rather uncertain outcome, but as we pick more marbles and calculate the fraction of blue marbles in our sample of picked marbles, we will get a better estimate of mu.

The importance of the Hoeffding Inequality is that it provides an upper bound for the likelihood that our estimate nu of the probability of blue marbles in the jar mu deviates by more than a fraction epsilon. Hoeffding's bound shows that the likelihood of exceeding a certain epsilon, is a function of that same epsilon squared, as well as the number of marbles in our sample, which we will call N.

The right hand side exponent e raised to a negative power will quickly become very small as N increases. In other words, as N increases, the likelihood of a significant difference between the observed fraction of blue marbles and the fraction of blue marbles in the bin becomes smaller and smaller. Significance in this regard, can be chosen by you by setting epsilon to an appropriate value.

Slide 3 Hoeffding for multiple draws

Later on we will see why this is important in the Theory of Learning. For now, let's explore what happens when we sample N marbles from a large number of different bins.

We sample N marbles from each jar assuming that every time we do this, we draw a sample that is representative of the true fraction of blue marbles mu. However, if you were to do this experiment for a large M, and for a moment assuming you can also observe the real fraction of blue marbles mu, you would find that every so often you will find staggering differences. For example you might find that you have picked a sample that only contains orange marbles.

So this of courses poses questions about the applicability of the Hoeffding bound to this new experiment where we repeat the original experiment M times. It doesn't look like the Hoeffding Bound should still apply, as, for large M, we will likely find that at least in one of our picks nu and mu deviate significantly, directly contradicting the Hoeffding Bound.

Slide 4 Bad events

To see what is going on, let's define the event that nu and mu differ by more than epsilon as a Bad event, B. Let's further use the subscript 1 through M to indicate this Bad event occurring for each of the M samples respectively.

With these definitions we can start thinking about the probability that this bad event will occur at least once in our M picks. As the individual M samples are all sampled from independent jars, we can say that the probability of at least one Bad event happening in M picks, is equal to the probability that the bad event happens in the first pick, or in the second pick, or in the third pick and so forth until the Mth pick. Using the union bound we can say that this probability is upper bound, or smaller than the sum of all the individual probabilities that the bad event will occur in pick 1 through M, which is the same as multiplying the probability of one bad event happening by M as all these individual probabilities are the exact same.

Slide 5 Hoeffding for large M

Clearly, if M is large, the Hoeffding bound is not very useful as it just says that a difference between mu and nu larger than epsilon is to be expected in one of the M jars. Especially for the right-hand side being larger than 1, the bound becomes meaningless as any probability can never be larger than one.

Slide 6 Some definitions (1)

Now we understand the Hoeffding bound and its limitations, let's look at why this Hoeffding Bound is so important in machine learning. For this, we need to introduce a few definitions. In machine learning we are interested in learning what function created the data that we have at our disposal. For example, if given a data set with characteristics, normally called features, of patients and medical tests, we want to determine if they have a certain disease or not. In supervised learning, we train machine learning models to approximate the relationship between features and disease classification, which we can think of as a mathematical function, as accurately as possible.

To achieve this goal, we will try a new hypothesis at every iteration until satisfied with the result. From one hypothesis to the next, we generally nudge the parameters of the hypothesis a small fraction. Such a family of hypotheses is also called a machine learning model. Examples are neural networks, linear regression or logistic regression, where every combination of model parameters would result in a new hypothesis.

Slide 7 Some definitions (2)

To determine when we are satisfied, we decide on an error measure. For classification this could for example be the fraction of misclassified data points in our training data set. Because we can only calculate this error on the training data points that we have in our sample, we call this the in-sample error.

All the points that are not in our sample, in other words all the data points that are somewhere out there and may need to be predicted by our model, we call the out-of-sample data points. We cannot define this error in the same way as this is an uncountable set of points, but we can formalise it as the probability that our hypothesis applied to such an out-of-sample data point does not agree with the actual class membership of that data point.

Slide 8 Conditions for learning

With these definitions we can define two conditions that will need to be met if want to be able to say that learning from data is possible:

The first is that our in-sample error is sufficiently small.

The second is that our out-of-sample error must be similar to our in-sample error. In other words, we must be confident that we can replicate the performance we have obtained on our training data, on new, yet unseen data.

The first condition can be observed during training, simply by looking at the performance of our machine learning algorithm on the training set. However, the second condition can never be observed, because we will never have access to all the data that our machine learning algorithm will see throughout its full service life. If this were the case, we might as well create a look up table.

So instead, we will need to create assurances that our out-of-sample error will be similar to our in-sample error, and this is where the Hoeffding bound comes in.

Slide 9 Hoeffding to bound |Ein-Eout|

If we think of orange marbles as data points where our current hypothesis is correct, and blue marbles as data points where our current hypothesis is incorrect, the data points in our sample will give rise to the in-sample error, and the data points in our jar will give rise to the out-of-sample error. We cannot see these data points, so the out-of-sample error is per definition unknown. However, we can now think of the difference between in-sample error and out-of-sample error for the current hypothesis, as bound by the Hoeffding inequality.

We had already seen that the right-hand side of this inequality, the upper bound on the probability of a bad event, is driven down as the number of samples N, that we have for training, increases. So this seems to suggest that all we need to do is take a large number of training samples and the Hoeffding bound will very snuggly limit the likely difference between our performance on training data and performance on data yet unseen.

Slide 10 Hoeffding for multiple hypotheses

However, now we need to think back of the experiment in which we sampled from M different jars where we saw that the right-hand side of the Hoeffding bound increased by a factor M.

The analogy with a machine learning algorithm is, that most machine learning algorithms will perform a large number of iterations to arrive at the winning hypothesis, that magic combination of model parameters that best describes or predicts the data. So when after having tried M different hypotheses, we observe good results on our sample, it is very appealing to claim victory and propose hypothesis M as our model that will do well on all yet unseen data. However, just like the M different picks from the jar with marbles invalidated the Hoeffding bound, so will the M draws from the hypothesis set invalidate the Hoeffding bound.

Slide 11 M is generally huge

Similar to the earlier experiment where we selected M samples from M jars, our first reaction should be that the Hoeffding bound which bounds the likelihood that our performance in-sample and out-of-sample are significantly different, should be increased by a factor M equal to the number of hypotheses that we could potentially try during training. Unfortunately this number is generally extremely high as the slightest change in a model parameter will result in a new hypothesis. With modern machine learning models using millions of parameters where each can take on millions of different values, the number of potential hypotheses is so large that the right-hand side of the Hoeffding bound becomes completely meaningless. This is shown in these plots of the Hoeffding bound, which compare the probability of the difference between in-sample and out-of-sample error being larger than 20%. For M=1, this is a vanishingly small number, for M=1,000,000 this is

approximately 670 which shows the Hoeffding bound is meaningless as probabilities are defined on the domain 0 to 1.

To put this in further perspective, a linear regression model, one of the simplest models you can think of, with one feature and a bias, would, on a computer with a floating point size of 32bits result in 18 quintillion, that is 18 times 10 to the power of 18, hypotheses! Clearly, this is not a fruitful way forward.

Slide 12 Dichotomies (1)

Fortunately, because machine learning per definition relies on a limited set of training data, we can simplify matters significantly. To see why this is the case, suppose you have a classification problem with 20 data points. 11 data points are of class 1 and 9 data points are of class 2. In the animation, you can see 6 equally good hypotheses on this dataset, but of course by making even smaller changes to the parameters from hypothesis to hypothesis, it is easy to see that there are many different hypotheses that result in the exact same classification of the data set.

Slide 13 Dichotomies (1)

Hypotheses can vary as much as they like, but as long as a new hypothesis does not result in at least one data point being classified differently, from the perspective of the data, all these hypotheses are one and the same. We call this reduced set of hypotheses, dichotomies. So for dichotomies it holds, that for one dichotomy to be different from any other dichotomy, it must classify at least 1 of the N points differently.

With this definition of dichotomies, we can now determine the maximum number of dichotomies that any model could create on a set of N data points. If a model is maximally flexible, it could create decision boundaries in such a way that every point in the dataset can be classified as either 0 or 1. In other words on N data points with each data point being able to be part of either of these 2 classes, we have a total of 2^N possible dichotomies. This means that M, which was potentially extremely large, has now been reduced to a number that is at maximum 2^N.

Slide 14 Hoeffding for maximum dichotomies

At this stage it is worth revisiting our Hoeffding plots, to see if, by replacing M by 2^N, we get a more meaningful bound. Before doing this, let's look at the derivative of this function. Now we see that, although for small epsilon, the derivative is positive, for some epsilon the derivative becomes negative. This is what we want as a negative derivative means that the function value will become smaller and this potentially starts becoming a useful upper bound. Unfortunately this happens for quite a large epsilon, which would mean that we can only bound the probability of the error being larger than roughly 0.58. The plots further illustrate this. In this plot the updated Hoeffding bound is plotted for various values of epsilon, but now as a function of N. What we see is that the bound increases rather than decreases with increasing N for all plotted values of epsilon. In this plot we pick two values of epsilon around the found value of 0.58 and see that, for values larger than 0.58, the bound indeed starts decreasing with increasing N. Although this is the type of behaviour we are looking for, given that our in-sample error and out-of-sample error are normalised to 1, an epsilon of 0.58 means that the final hypothesis is likely to perform very badly on yet unseen data. Hence, with this equation we would only be able to provide an upper bound for the probability that our machine learning model will perform quite badly.

These results suggest that we need to find a way to reduce the multiplier in the Hoeffding bound further. The equations also suggest that it would be useful if we can express this multiplier as a

function of N in such a way that the multiplier grows slower as N increases than the exponential decreases as N increases, as this would result in an overall function that decreases as N increases. The results would be something that is similar to the behaviour of our original Hoeffding bound.

Slide 15 A quick recap

At this stage it is worth considering once more what this multiplier represents. Initially, we had a factor M, which was equal to the potentially huge number of different hypotheses that we can create with our machine learning model. We then reduced this to a maximum of 2^N after realising that on N data points, only the 2^N hypotheses that result in different dichotomies matter. However, we just saw that the function 2^N still grows too fast to be suppressed by the exponent in the Hoeffding bound. Hence our quest is to find a multiplier that grows slower as N increases such that the negative exponent starts dominating and pushing the whole right-hand side of the equation towards zero from a certain N onwards. In a few slides we will see that finding such a multiplier is directly related to restricting the number of hypotheses that are good candidates to represent the data as the dataset size N increases. In other words, an increasing N should reduce the flexibility of the set of hypotheses that are good candidates for representing the N data points.

Slide 16 Relating model complexity to N (1)

Let's see if this insight, that an increasing number of data points N, should in some way reduce the flexibility of the hypothesis set, makes sense. To see this, imagine that you have the completely fictional 'linear spikes' model, which creates a linear decision boundary with spikes into the regions occupied by class 1 to envelope stray points in class 0. For this situation, with a limited number of data points, this model finds a perfect decision boundary, which contains a few spikes to capture the data points of class 0 that lie in class 1 and vice versa.

But let's consider if this decision boundary is likely to be a good choice when considering the new data. Several of the points captured by the spikes seem quite isolated, so we may need to consider whether these are outliers in our data, or were simply mislabelled. We would find this out quite quickly if we had some extra data that could provide more evidence as to the validity of the spikes in our decision boundary.

Slide 17 Relating model complexity to N (2)

As you can see, the extra data points show that most of the spikes were unwise choices and that the extra data points have forced the learning algorithm to consider a simpler model with fewer spikes.

Slide 18 Possible dichotomies (2^N max)

We can formalise this intuition by reconsidering the dichotomies introduced earlier. Rather than thinking of a particular dataset of N points, we now think of a constellation of N data points, in which each data point can take on either class membership. As we already know, such a constellation of N data points results in a maximum of 2^N dichotomies. We now wish to establish how many dichotomies can be achieved by our learning algorithm on this constellation and further want to relate this to the number of points in our constellation, N. Moreover, we wish to find the absolute maximum number of dichotomies possible, so rather than providing any random constellation, we will provide constellations such that we find the absolute maximum number of dichotomies. The maximum number of dichotomies, as a function of N, that can be realised by a model is called the growth function of that model.

Slide 19 Shattering the data set

And if a model or hypothesis set can realise all 2^N dichotomies, the model is said to 'shatter' the data for those N points.

Slide 20 Shattering and breakpoints (for classification)

Let's see what this growth function might look like for the perceptron algorithm as a linear classifier that can create one straight decision boundary between any of the points. For a constellation of one point, the perceptron can create any of the 2^1 dichotomies, and hence the perceptron's growth function for N=1 equals 2. For two points, the perceptron can create any of the 2^2 dichotomies, so the growth function of a perceptron for N=2 equals 4. If we add a further point, we similarly see that the perceptron can create all 2^3 dichotomies, resulting in a growth function of 8 for N=3. However, for 4 points, we see that there is no constellation that results in the perceptron being able to realise all 16 dichotomies. In fact, no matter how hard you try, you will not be able to find more than 14 possible dichotomies. The two dichotomies that cannot be realised are shown here and the result is that the perceptron growth function for N=4 equals 14.

As we see in this example, there may come a point in time, or rather data points N, where the model can no longer shatter the data set, no matter what constellation you try. This N is called a break point of the model. And it can be shown that, if N is a break point of the model, then N+1 is also a break point of the model. In other words, once a break point is found, the number of dichotomies realisable by the model on N points or more, will reduce relative to N. Another way of looking at this, is that the growth function, which may well be equal to 2^N for some values of N, will start growing slower than with a factor of 2 for each extra datapoint from a certain value of N onwards.

And this is a really good result! Because this shows, that instead of using 2^N in the Hoeffding bound, we can use the growth function as a tighter upper bound on the number of dichotomies realisable by the model for N data points. Hence, there is now a better chance that the exponent will decrease fast enough to cancel the effect of the growth function.

Slide 21 How can we generalise these results?

Growth functions vary significantly from model to model and as a result our updated bound is still very model dependent. Hence, the last job left to do before we can present our final updated Hoeffding bound, is to replace the growth function with something a little more generic. In mathematics, generic expressions come with a price. Generally, that price is that, just like in the original Hoeffding bound, we cannot give exact values, but can only say what the maximum possible value is. Hence, we will look for an upper bound on the growth function.

With some mathematics, which we will choose to skip, it is possible to show that, if a hypothesis set has a break point for N=k and k is finite, that the growth function is always smaller than:

$$m_H(N) \leq \sum_{i=0}^{k-1} \binom{N}{i}$$

Which again, without proof, can be further simplified to:

$$m_H(N) \leq N^{k-1} + 1 = N^{d_{VC}} + 1$$

This equation says that the growth function is always smaller than or equal to the number of data points N raised to the lowest break point minus 1 plus the constant 1. K-1 is equal to the largest N for which the hypothesis set shatters a constellation of N points, and this N can be thought of as expressing the complexity of the hypothesis set. A small value suggests that the hypothesis set will fail to shatter a small constellation and hence its ability to perform correct classification on a small dataset is already limited. Larger values mean the hypothesis set is able to shatter a larger constellation of N points and hence it is likely to be able to better classify any larger data set. This is called the complexity of the hypothesis set and in recognition of the seminal work performed by Vapnik and Chervonenkis, k-1 is known as the VC dimension of the model. Similarly, the updated Hoeffding bound with the upper bound on the growth function and some further mathematical housekeeping is called the VC bound.

Slide 22 Sample complexity (1)

And with that we have almost come to one of the best-known results of the VC bound. If we plot both the Hoeffding bound and the VC bound, we see that the VC bound is a very loose bound compared to the Hoeffding bound. In fact, in this particular example for a model with a VC dimension of 3 and an N corresponding to the number of training data points of 800, the VC bound is still meaningless. We know that, as we increase the number of data points N, the VC bound will become tighter and will eventually provide a meaningful bound on the likely difference between Ein and Eout, but how many data points N, do we need for this? To establish exactly how much data the VC bound predicts we need, we first introduce the tolerance delta as the left-hand side of the VC bound. Delta represents the probability that the epsilon is exceeded, so we can think of 1-delta as our confidence that the difference between in-sample and out-of-sample error will not exceed the chosen epsilon.

We can rewrite this equation to solve for N. However, we see that the expression for N contains N itself. For this reason, we use an iterative approach to find N, the number of data points required for a certain epsilon and a certain delta. For example, let's choose epsilon 0.1 which means we will accept a maximum difference between in-sample error and out-of-sample error of 10%. And let's choose delta 0.1 as well, which means that we accept that there is a 10% chance that the epsilon of 0.1 will be exceeded. We also choose a VC dimension, three in this first case and we just guess that the required N is 1000. Filling these values into the found equation for N, yields an N of 21,193, so clearly our first guess was far too low. Given the plot of the VC bound for a model with VC dimension 3 and 800 data points, this is not entirely unexpected, but we had to start somewhere. To further home in on the required N, we can fill the found N into the same equation to get an updated estimate, and we find that the new estimate for the minimum N to obtain the chosen epsilon and delta is now 28,522. As there is still quite a big difference between the N we plugged in to the equation and the N that rolled out, we can do another iteration and we find an N of 29,234. We could continue this another few times and we will find that the difference between the N we plug in and the N that rolls out becomes smaller and smaller. In other words, we are converging to the real minimum value of N and this turns out to be roughly 30,000 in this case.

To understand the relationship between VC dimension and required training data, we repeat this process for a VC dimension of 4, and of 5. Each time, we start the process with the final N from the previous step. This makes perfect sense as we know that a higher VC dimension will always result in an increased demand for training data.

And what we find is that there is quite a satisfying relationship between the VC dimension of a model and the amount of training data required. We find that the VC bound predicts that we need a number of data points that is roughly equal to the VC dimension of the model multiplied by 10,000.

Slide 23 Sample Complexity (2)

Let's reflect on this staggering result. After extensive use of quite tough mathematics, we end up with a very generic and simple result: no matter what machine learning model we use, as long as we know its VC dimension we can provide assurances as to the performance of the trained model on yet unseen data simply by ensuring our data training set is large enough.

Coming up with such generic results has cost us a significant amount and in a way this is good news. Due to the use of a number of higher bounds, the amount of training data required is in reality far less than predicted by the VC bound. The rule of thumb is that you require a training data set size of ten times the VC dimension instead of 10,000. This is a rule of thumb that you will use time and again when tackling a machine learning problem.

Slide 24 Summary

To summarise our derivation of the VC bound:

We started with the Hoeffding bound, a generic result from probability theory, and applied this to provide an upper limit on the probability that the in-sample error and the out-of-sample error will deviate by more than a certain epsilon. We did this, as we thought this might provide us assurances as to the performance of our trained model on yet unseen data.

We noticed that, as the training data set size N is increased, the probability of the in-sample error and out-of-sample error deviating by more than epsilon, decreases.

This was the good news, but we also saw that the Hoeffding bound cannot be applied directly to machine learning as the Hoeffding bound is not valid when we create multiple estimates of the in-sample error for multiple hypotheses. Initially we saw, that in a worst-case scenario, this would result in having to multiply the right-hand side of the Hoeffding bound by M, the number of hypotheses in a model.

Unfortunately this factor M is so large in any practical scenario that it results in the Hoeffding bound not being a practical bound at all, so we started looking at ways to find a replacement for M that would allow us to create a more restrictive bound. Through the definition of dichotomies, we came up with the growth function as the effective number of hypotheses that can be created by a model on a dataset of size N. We saw that this growth function, based on the definition of dichotomies, was at most 2^N, but trying this in our bound did not yet yield satisfactory results, so we started to see if we could further constrain the growth function.

We saw that, for small N, models may well be able to realise all 2^N dichotomies, in which case we said that the model shattered the data set, but in general, for a certain N this is no longer the case.

When this is no longer the case, the growth function starts increasing slower than with a factor of 2 for each increment in N and we defined the highest N for which the model can still shatter the dataset as its VC dimension. This is a logical definition, as the VC dimension now delineates the largest N for which the model is still maximally flexible. And because flexibility in a model is directly related to its ability to model complex datasets, we also called this the complexity of the model. Some further mathematics, including some further upper bounding, resulted in a polynomial expression for the growth function in N, on the condition that the VC dimension is finite. And with

this expression, we finally found a version of the Hoeffding bound that will constrain the probability that the in-sample error and the out-of-sample error deviate by more than a chosen epsilon, in a meaningful way. This end result of our hard work we called the Vapnik Chervonenkis, or VC bound, in honour of the two mathematicians who did all this hard work for us.

And whilst we found that this new bound is still very loose in theory, it provides us with two very important conclusions:

First of all, the VC bound proves that learning from data is really possible.

And second of all, and perhaps this is the more surprising result, the VC bound shows that there is a very simple relationship between the VC dimension of a model, and the amount of data required to say that the model has truly learned with considerable confidence. And in practice, this results in the rule of thumb that the amount of required data is 10 times the VC dimension of the model used.